

Universidad del Valle

Facultad de Ingeniería

Escuela de Ingeniería de Sistemas y Computación

Inteligencia Artificial

Proyecto 1

Robotaxi Zoon

Estudiantes:

Alexander Garzon Mariaca (2510017)

Juan Carlos Cruz Muñoz (1824389)

Estiven Andres Martinez Granados (2179687)

Profesor:

Oscar Fernando Bedoya Leiva

Semestre:

2026-I

Repositorio del proyecto:

<https://github.com/Juan-Carlos-Cruz/robotaxi-zoon/tree/main>

1 de junio de 2026

Índice

1. Descripción del problema	2
2. Formulación del problema como búsqueda	3
2.1. Estados	3
2.2. Operadores	3
2.3. Meta	4
2.4. Función costo de ruta	4
2.5. Árbol de búsqueda y espacio de estados	4
3. Correspondencia entre teoría e implementación	4
4. Algoritmos de búsqueda no informada	6
4.1. Caso reducido para visualizar el árbol	6
4.2. Búsqueda preferente por amplitud	7
4.3. Búsqueda de costo uniforme	8
4.4. Búsqueda preferente por profundidad evitando ciclos	9
5. Búsqueda informada y diseño de la heurística	10
5.1. Por qué esta heurística tiene sentido en este problema	11
5.2. Búsqueda avara	12
5.3. Algoritmo A^*	13
5.4. Por qué la heurística de A^* es admisible	14
5.5. Control de estados repetidos en avara y A^*	14
6. Resumen de propiedades de los algoritmos	15
7. Interfaz y ejecución	15
8. Conclusiones	16

1. Descripción del problema

El proyecto se desarrolla en una ciudad representada como una cuadrícula de 10×10 . En ella, un robotaxi Zoox parte desde una posición inicial y debe recoger a todos los pasajeros distribuidos en el mapa antes de dirigirse al único punto de destino. El entorno incluye casillas libres, muros que bloquean el paso y zonas de flujo vehicular alto que pueden recorrerse, pero con un costo mayor.

Por tanto, el objetivo no es simplemente llegar al destino, sino recoger primero a todos los pasajeros y finalizar luego en la posición indicada. Bajo el supuesto de que todos los pasajeros se dirigen al mismo punto de llegada, la **Figura 1** resume el escenario base del proyecto y permite identificar con claridad los elementos principales del mapa.

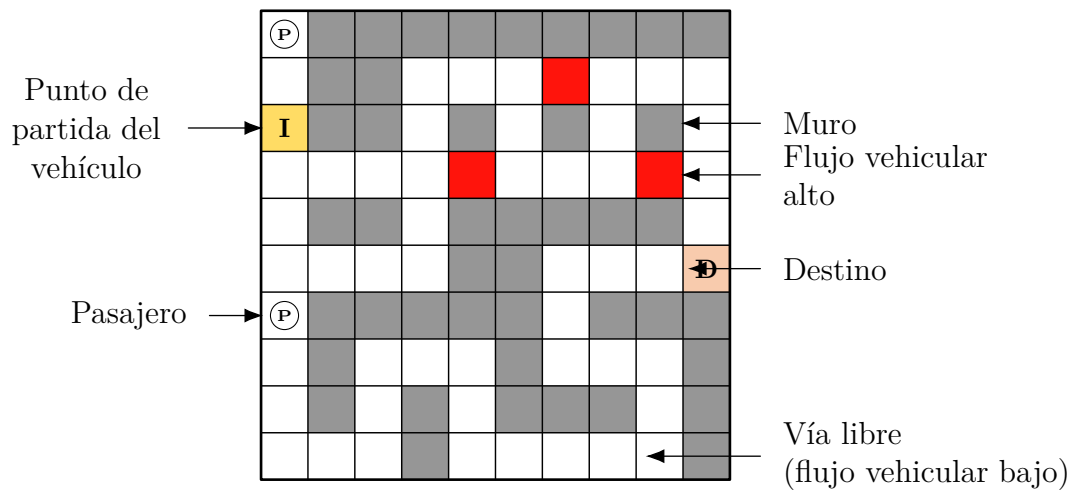


Figura 1: Mapa base del problema Robotaxi Zoox.

El vehículo solo puede desplazarse en las cuatro direcciones cardinales: arriba, abajo, izquierda y derecha. El costo del desplazamiento depende del tipo de casilla a la que se llega:

- Vía libre, inicio, pasajero y destino tienen costo 1.
- Flujo vehicular alto tiene costo 7.

Estos costos no cambian por llevar pasajeros; dependen solo del tipo de casilla a la que se entra. Además, la cantidad de pasajeros puede variar entre ambientes de prueba, aunque siempre existe al menos uno.

En la representación del mapa se usan los siguientes valores:

- 0: casilla libre (flujo vehicular bajo);
- 1: muro;

- 2: punto de partida del vehículo;
- 3: casilla con flujo vehicular alto;
- 4: pasajero;
- 5: destino.

Si el robot entra en una casilla con pasajero, lo recoge automáticamente. Con esta representación del entorno, cada algoritmo busca una ruta válida siguiendo su propio criterio de exploración.

2. Formulación del problema como búsqueda

En los problemas de búsqueda, lo habitual es describir el problema mediante **estados**, **operadores**, **meta** y **función costo de ruta**. Bajo ese esquema, el caso del robotaxi puede expresarse como un problema formal de búsqueda sobre un árbol o grafo de estados.

2.1. Estados

El estado del problema queda definido por dos componentes: la celda actual del taxi y el conjunto de pasajeros que ya han sido recogidos.

Dicho de forma directa, el estado puede leerse como:

- posición actual del vehículo;
- conjunto de pasajeros ya recogidos.

Así, el estado inicial ubica al taxi en la casilla de inicio sin pasajeros recogidos, y el estado meta corresponde al taxi en el destino con todos los pasajeros ya recogidos.

Guardar solo la posición no basta, porque una misma casilla puede representar situaciones distintas según los pasajeros recogidos. Como el mapa y la cantidad de pasajeros son finitos, el espacio de estados también lo es.

2.2. Operadores

Los operadores corresponden a los cuatro movimientos ortogonales permitidos por el entorno: mover a la derecha, arriba, abajo o izquierda.

Cada operador genera un nuevo estado solo si la casilla de llegada es válida y transitable. Si el movimiento lleva a un muro, no se puede aplicar. Si el taxi entra en una casilla con pasajero, el nuevo estado conserva la nueva posición y además agrega ese pasajero al conjunto de pasajeros recogidos.

2.3. Meta

La meta se cumple únicamente cuando el robotaxi llega al destino después de recoger a todos los pasajeros del mapa.

2.4. Función costo de ruta

Esta función suele llamarse g , y aquí representa simplemente el costo acumulado de la ruta hasta el nodo actual.

En este problema:

- entrar a una vía libre, a la casilla inicial, a una casilla con pasajero o al destino cuesta 1;
- entrar a una casilla de flujo vehicular alto cuesta 7.

Entonces este no es un problema de costo uniforme. Ese detalle explica por qué **bfs** no garantiza la mejor solución en costo, mientras que **ucs** y **A*** sí pueden hacerlo si se cumplen las condiciones teóricas correspondientes.

2.5. Árbol de búsqueda y espacio de estados

En un árbol de búsqueda, el **estado inicial** es el **nodo raíz** y el algoritmo, en cada paso, escoge un **nodo hoja** para expandirlo.

Conviene distinguir dos niveles:

- el **estado del problema**, que solo necesita posición y pasajeros recogidos;
- el **nodo del árbol de búsqueda**, que además guarda padre, profundidad, costo acumulado, valor heurístico y evaluación total.

Esta separación es importante porque varios nodos del árbol pueden representar el mismo estado abstracto, pero haber sido alcanzados por rutas distintas y con costos diferentes. Precisamente allí aparecen los conceptos de **visitados** y **best_g**.

3. Correspondencia entre teoría e implementación

La organización del código sigue de forma bastante fiel la formulación anterior, aunque esa relación está repartida entre varios archivos. La Tabla 1 resume esa correspondencia.

Concepto teórico	Interpretación	Representación en código
Estado	Posición actual del taxi y conjunto de pasajeros recogidos	atributos <code>posicion</code> y <code>pasajeros_recogidos</code>
Nodo del árbol	Estado más información auxiliar para la búsqueda	clase <code>Node</code> : padre, profundidad, costo, heurística y evaluación
Operadores	Mover el taxi en cuatro direcciones cardinales	método <code>get_vecinos()</code>
Meta	Llegar al destino con todos los pasajeros recogidos	condición en <code>wrapper.py</code>
Función costo de ruta	Suma de costos de las casillas recorridas	atributo de costo y método <code>costo_movimiento()</code>
Frontera	Conjunto de hojas pendientes por expandir	cola FIFO, pila LIFO o cola de prioridad
Control de repetición	Evitar expandir estados equivalentes sin perder retornos válidos	<code>visitados</code> o <code>best_g</code> usando posición y pasajeros

Cuadro 1: Correspondencia entre la formulación teórica y la implementación.

Eso se ve en los siguientes fragmentos:

Listing 1: Información guardada en Node

```
self.posicion = posicion
self.pasajeros_recogidos = pasajeros_recogidos
self.padre = padre
self.g = g
self.profundidad = (padre.profundidad + 1) if padre else 0
```

Listing 2: Meta y control de repetición en wrapper.py

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)

if estado in visitados:
    continue

if nodo.posicion == grid.destino and nodo.pasajeros_recogidos == pasajeros_totales:
    return ...
```

Listing 3: Generación de sucesores y actualización del estado

```
costo = nodo.g + self.costo_movimiento(nueva_fila, nueva_columna)
nuevos_pasajeros = set(nodo.pasajeros_recogidos)

if self.matriz[nueva_fila][nueva_columna] == Grid.PASAJERO:
    nuevos_pasajeros.add((nueva_fila, nueva_columna))
```

Ese último fragmento muestra que la transición no solo cambia la posición: también puede modificar el conjunto de pasajeros recogidos.

4. Algoritmos de búsqueda no informada

Una búsqueda no informada funciona a ciegas: no dispone de información adicional que indique qué tan cerca está un nodo de la meta. Solo usa la formulación del problema y, en el caso de costo uniforme, el costo acumulado de la ruta.

Los algoritmos `bfs`, `dfs` y `ucs` comparten el mismo bucle general de expansión. Lo único que cambia es el criterio con el que se elige el siguiente nodo de la frontera:

- `bfs`: usa una cola FIFO, de modo que el primer nodo en entrar es el primero en salir.
- `dfs`: usa una pila LIFO, de modo que el último nodo en entrar es el primero en salir.
- `ucs`: usa una cola de prioridad, de modo que primero sale el nodo con menor costo acumulado.

En el código, esa diferencia se implementa dentro de `algoritmosBusqueda/noinformada/wrapper.py` y el control de repetición se hace sobre el estado compuesto (posición, pasajeros_recogidos).

Además, el orden de expansión entre hijos (y por tanto el desempate en BFS/DFS) está determinado por el orden de movimientos en `mundo/models/grid.py`:

Listing 4: Orden de movimientos y generación de vecinos

```
movimientos = [(0, 1), (-1, 0), (1, 0), (0, -1)]

for delta_fila, delta_columna in movimientos:
    ...
    vecinos.append(Node(..., g=costo))
```

4.1. Caso reducido para visualizar el árbol

Para comparar con claridad las búsquedas no informadas e informadas, se usa el caso reducido de la **Figura 2**. Desde el estado inicial existen dos decisiones inmediatas: ir a

la derecha por una casilla de tráfico alto o bajar por una casilla de costo bajo hacia el pasajero.

I	T		D
	M		
P			

Figura 2: Caso reducido usado para ilustrar el comportamiento de las estrategias de búsqueda. I = inicio, P = pasajero, D = destino, T = flujo alto, M = muro.

4.2. Búsqueda preferente por amplitud

La estrategia expande primero el nodo raíz, luego los nodos de profundidad 1, después los de profundidad 2, y así sucesivamente. En el código esto se implementa con una cola FIFO.

Listing 5: Implementación de BFS (frontera FIFO)

```
frontera = deque([nodo_inicial])

def pop():
    return frontera.popleft()
```

El control de ciclos para `bfs` se hace con el estado compuesto, evitando reexpandir el mismo par (posición, pasajeros_recogidos):

Listing 6: Control de ciclos en BFS

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)
if estado in visitados:
    continue
visitados.add(estado)
```

En el árbol reducido, el orden de expansión entre los dos hijos de la raíz coincide con el orden de movimientos mostrado antes; por eso, aunque ambos están al mismo nivel, se expande primero el que corresponde a “derecha”.

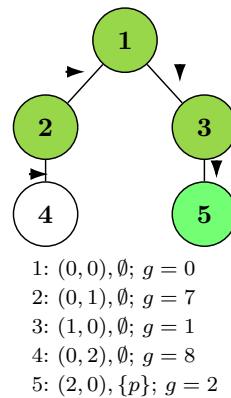


Figura 3: Árbol parcial de expansión de **bfs** en el caso reducido. El número de cada nodo indica el orden de expansión observado en la implementación y las flechas muestran el movimiento aplicado.

La **Figura 3** muestra que **bfs** primero expande el hijo derecho del nodo inicial porque ambos hijos están en la misma profundidad y el desempate lo resuelve el orden de operadores definido en `Grid.get_vecinos()`. Esto coincide con la explicación teórica de que, dentro de un mismo nivel, el orden de aplicación de operadores sí importa.

Teóricamente, **bfs** es completa si la ramificación es finita, pero solo garantiza optimalidad cuando todos los pasos cuestan lo mismo. Como hay casillas con costos distintos, **bfs** debe entenderse como una estrategia que busca rutas con menos movimientos, no necesariamente con menor costo total.

4.3. Búsqueda de costo uniforme

La búsqueda de costo uniforme expande el nodo con menor costo acumulado, sin importar su profundidad. En la implementación, esto se logra con una cola de prioridad basada en `heapq`.

Listing 7: Implementación de UCS (cola de prioridad por g)

```
frontera = []
heapq.heappush(frontera, (nodo_inicial.g, contador, nodo_inicial))

def pop():
    return heapq.heappop(frontera)[2]
```

En **ucs**, el control de ciclos es el mismo que en **bfs/dfs** porque el estado incluye los pasajeros recogidos:

Listing 8: Control de ciclos en UCS

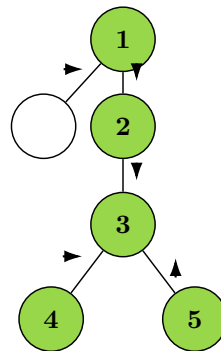
```
estado = (nodo.posicion, nodo.pasajeros_recogidos)
if estado in visitados:
```

```

    continue
    visitados.add(estado)

```

Por eso, en la **Figura 4** el estado con costo $g = 7$ queda pendiente: la prioridad se define por g y no por el orden de generación.



1: $(0, 0), \emptyset; g = 0$
 pend.: $(0, 1), \emptyset; g = 7$
 2: $(1, 0), \emptyset; g = 1$
 3: $(2, 0), \{p\}; g = 2$
 4: $(2, 1), \{p\}; g = 3$
 5: $(1, 0), \{p\}; g = 3$

Figura 4: Árbol parcial de expansión de **ucs** en el caso reducido. El número de cada nodo indica el orden de expansión observado en la implementación y las flechas muestran el movimiento aplicado.

El árbol permite ver el efecto más importante: aunque el estado $(0, 1)$ fue generado antes, no se expande de inmediato porque entrar allí cuesta 7. En cambio, se prioriza la rama inferior cuyo costo acumulado es menor. Esta conducta reproduce exactamente el criterio teórico de costo uniforme.

Como todos los costos del problema son positivos, **ucs** mantiene su propiedad: si existe solución, la encuentra, y además la solución encontrada es óptima respecto al costo total de la ruta.

4.4. Búsqueda preferente por profundidad evitando ciclos

La búsqueda preferente por profundidad siempre intenta expandir el nodo más profundo disponible en la frontera. La pila LIFO hace que el último sucesor generado sea el primero en explorarse.

Listing 9: Implementación de DFS (frontera LIFO)

```

frontera = [nodo_inicial]

def pop():
    return frontera.pop()

```

El control de ciclos en `dfs` también se apoya en el estado compuesto para evitar bucles:

Listing 10: Control de ciclos en DFS

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)
if estado in visitados:
    continue
visitados.add(estado)
```

En el caso reducido, esto explica por qué la expansión sigue una sola rama antes de volver a los nodos pendientes: el `pop()` devuelve siempre el último vecino insertado.

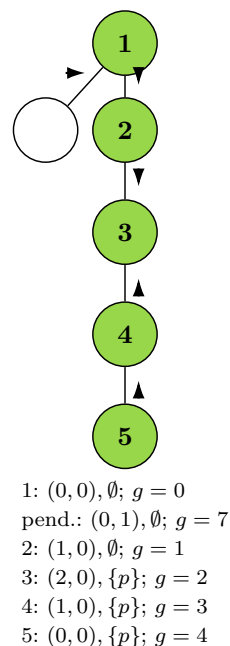


Figura 5: Árbol parcial de expansión de `dfs` en el caso reducido. El número de cada nodo indica el orden de expansión observado en la implementación y las flechas muestran el movimiento aplicado.

La **Figura 5** muestra cómo la búsqueda profundiza por una sola rama antes de volver a los estados pendientes.

En este caso el detalle relevante es el control de ciclos sobre el estado compuesto (`posicion`, `pasajeros_recogidos`). Eso evita que la búsqueda quede atrapada devolviéndose indefinidamente, pero sigue permitiendo regresar a una casilla si el conjunto de pasajeros ya cambió. Desde la teoría, esta es la versión correcta de **evitar ciclos**, que es más fuerte que simplemente evitar el regreso inmediato.

5. Búsqueda informada y diseño de la heurística

La heurística se entiende como una estimación de lo que todavía falta para completar la solución. Esa idea se adapta aquí con un matiz importante: cuando aún faltan pasajeros,

la meta global no puede alcanzarse directamente; antes es obligatorio llegar al menos a uno de los pasajeros pendientes.

5.1. Por qué esta heurística tiene sentido en este problema

La heurística base es la distancia Manhattan. En palabras simples, mide cuántas filas y cuántas columnas separan dos celdas, y suma ambas cantidades.

Su uso es natural por tres razones:

- el vehículo se mueve únicamente en direcciones ortogonales;
- cada desplazamiento cuesta al menos 1;
- los muros y el tráfico alto solo pueden aumentar el costo real, nunca reducirlo por debajo de la distancia Manhattan.

Sin embargo, la heurística no debe mirar siempre al destino. Si todavía faltan pasajeros, una solución válida necesariamente debe pasar primero por alguno de ellos.

La lógica es la siguiente:

- mientras falten pasajeros, usamos la distancia Manhattan al pasajero pendiente más cercano;
- cuando ya no faltan pasajeros, usamos la distancia Manhattan desde la posición actual hasta el destino.

Formalmente:

- si faltan pasajeros, $h(n) = \min_{p \in P_{\text{falt}}} (|x - x_p| + |y - y_p|)$;
- si no faltan pasajeros, $h(n) = |x - x_d| + |y - y_d|$.

Eso hace que la heurística siga la estructura real del problema: primero recoger pasajeros, después cerrar la ruta en el destino.

La implementación sigue exactamente esa idea:

Listing 11: Heurística implementada

```
def calcular_heuristica(nodo, destino, pasajeros, meta_pasajeros):
    :
    if nodo.pasajeros_recogidos == meta_pasajeros:
        return heuristica_manhattan(nodo.posicion, destino)

    pasajeros_faltantes = [p for p in pasajeros if p not in nodo.
                           pasajeros_recogidos]
    return min(heuristica_manhattan(nodo.posicion, p) for p in
               pasajeros_faltantes)
```

5.2. Búsqueda avara

La búsqueda avara selecciona el nodo con mejor valor heurístico. En términos prácticos, expande el nodo que **parece** más cercano a la meta sin tomar en cuenta el costo ya pagado.

Listing 12: Implementación de búsqueda avara (prioridad por h)

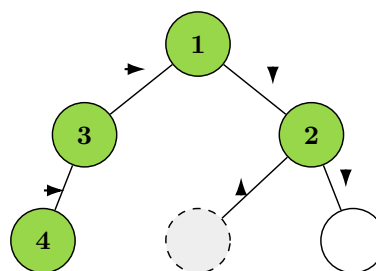
```
heapq.heappush(frontera, (nodo_inicial.h, contador, nodo_inicial)
)

for vecino in grid.get_vecinos(nodo):
    vecino.h = calcular_heuristica(vecino, destino, pasajeros,
    meta_pasajeros)
    prioridad = vecino.h
    heapq.heappush(frontera, (prioridad, contador, vecino))
```

Para evitar ciclos, la búsqueda avara marca el estado compuesto como visitado:

Listing 13: Control de ciclos en búsqueda avara

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)
if estado in visitados:
    continue
visitados.add(estado)
```



1: (0,0), $\emptyset$; $h = 2$
 3: (0,1), $\emptyset$; $h = 3$
 2: (1,0), $\emptyset$; $h = 1$
 4: (0,2), $\emptyset$; $h = 4$
 desc.: (0,0), $\emptyset$; $h = 2$
 pend.: (2,0), $\{p\}$; $h = 5$

Figura 6: Árbol parcial de expansión de la búsqueda avara en el caso reducido. El número de cada nodo indica el orden de expansión observado en la implementación y las flechas muestran el movimiento aplicado.

La **Figura 6** muestra que la búsqueda avara puede posponer el estado donde ya se recogió el pasajero porque desde allí la distancia al destino aún es grande. En otras palabras, la heurística por sí sola ignora el costo ya invertido.

5.3. Algoritmo A*

El algoritmo A* combina dos ideas: lo que ya costó llegar al nodo actual y lo que todavía se estima que falta.

Listing 14: Implementación de A* (prioridad por $f = g + h$)

```
heapq.heappush(frontera, (nodo_inicial.f, contador, nodo_inicial)
)

for vecino in grid.get_vecinos(nodo):
    vecino.h = calcular_heuristica(vecino, destino, pasajeros,
    meta_pasajeros)
    vecino.f = vecino.g + vecino.h
    prioridad = vecino.f
    heapq.heappush(frontera, (prioridad, contador, vecino))
```

En A*, el control de ciclos usa `best_g` para conservar la mejor ruta a cada estado:

Listing 15: Control de ciclos en A* con `best_extunderscore g`

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)
if estado in best_g and best_g[estado] <= nodo.g:
    continue
best_g[estado] = nodo.g
```

Con la notación usual, A* ordena usando $f = g + h$. Eso recoge la idea central: **ucs** empuja la búsqueda a reducir el costo acumulado, la avara intenta reducir la estimación restante, y A* balancea ambas cantidades para priorizar el nodo con menor costo total estimado.

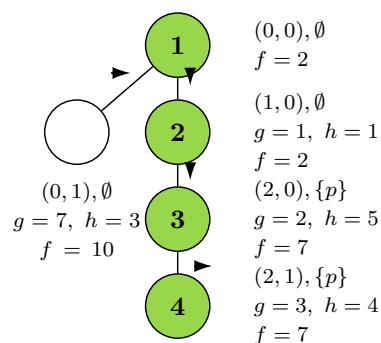


Figura 7: Árbol parcial de expansión de A* en el caso reducido. Las flechas muestran el movimiento aplicado en cada expansión.

A diferencia de la **Figura 6**, la **Figura 7** muestra que A^* no pospone el estado donde ya se recogió el pasajero, porque combina lo recorrido con lo que todavía falta.

5.4. Por qué la heurística de A^* es admisible

Para usar A^* con seguridad, basta verificar una idea muy simple: la heurística no debe exagerar el costo que aún falta por recorrer.

El mapa solo permite moverse arriba, abajo, izquierda y derecha, y cada paso cuesta como mínimo 1. Por esa razón, la distancia Manhattan siempre sirve como una estimación conservadora: puede quedarse corta, pero no puede decir que falta más de lo que realmente cuesta llegar.

La intuición se entiende en dos situaciones:

- **Si todavía faltan pasajeros**, la heurística toma la distancia al pasajero pendiente más cercano. Como cualquier solución válida tiene que llegar primero a algún pasajero antes de terminar la tarea, esa estimación no puede ser mayor que el costo real que aún falta.
- **Si ya no faltan pasajeros**, la heurística toma la distancia Manhattan hasta el destino. De nuevo, esa distancia tampoco supera el costo real restante, porque llegar al destino exige al menos esa cantidad de movimientos.

En otras palabras, para cualquier estado se cumple que la heurística nunca sobreestima el costo pendiente, es decir, $h(n) \leq C^*(n)$.

Eso es justamente lo que significa que la heurística sea **admisable**. Gracias a eso, A^* conserva su propiedad más importante: cuando encuentra una solución, esa solución es óptima en costo.

5.5. Control de estados repetidos en avara y A^*

En la implementación, la búsqueda avara usa un conjunto `visitados` sobre el estado compuesto (`posicion`, `pasajeros_recogidos`). A^* emplea un criterio más fino: mantiene en `best_g` el mejor costo conocido para cada estado y descarta solo las repeticiones dominadas por una ruta mejor.

Esta decisión conecta bien la teoría con la práctica:

- un mismo estado puede aparecer varias veces en el árbol de búsqueda;
- no todas esas apariciones merecen expandirse;
- en A^* solo se conserva la versión útil de un estado, es decir, la de menor costo acumulado.

6. Resumen de propiedades de los algoritmos

El Cuadro 2 resume las propiedades clásicas y su interpretación en el contexto del robotaxi.

Algoritmo	Criterio de expansión	Propiedad general	Interpretación
Amplitud	menor profundidad	completa; óptima solo si todos los pasos cuestan lo mismo	encuentra rutas cortas en número de acciones, pero no necesariamente de menor costo porque el mapa tiene casillas de costo 7
Costo uniforme	menor costo acumulado	completa y óptima con costos positivos	funciona como referencia exacta de costo óptimo en el mapa del robotaxi
Profundidad evitando ciclos	mayor profundidad	no óptima; en árboles infinitos no es completa en general	aquí termina porque el espacio de estados es finito y se usa control de ciclos, pero puede producir soluciones costosas
Avara	mejor valor heurístico	no óptima; sin control de ciclos no es completa en general	en esta implementación termina por el control de estados, pero su criterio puede tomar decisiones miopes
A*	mejor suma entre costo recorrido y estimación restante	completa y óptima si la heurística es admisible	combina el costo real recorrido con la estimación restante y conserva optimalidad para el problema planteado

Cuadro 2: Resumen de propiedades de los algoritmos y su interpretación.

7. Interfaz y ejecución

El proyecto no se limita a resolver el problema en abstracto. También ofrece una interfaz gráfica que permite:

- cargar un mapa desde archivo;
- visualizar el ambiente urbano;
- seleccionar la familia de búsqueda y el algoritmo concreto;
- animar el camino solución;
- reportar profundidad, nodos expandidos, costo, tiempo de búsqueda y heurística final cuando aplica.

La parte práctica permite observar en el mapa cómo se comporta cada búsqueda.

8. Conclusiones

- **Amplitud:** ventaja: encuentra soluciones con pocos movimientos; desventaja: no minimiza bien el costo cuando hay casillas caras.
- **Profundidad:** ventaja: puede avanzar rápido por una rama y usar poca memoria; desventaja: puede alejarse mucho de una buena solución y producir rutas costosas.
- **Costo uniforme:** ventaja: garantiza la ruta de menor costo; desventaja: suele expandir más nodos y tardar más que otras estrategias.
- **Avara:** ventaja: orienta la búsqueda hacia lo que parece prometedor y normalmente reduce exploración; desventaja: puede tomar decisiones miopes porque no considera el costo ya acumulado.
- **A*:** ventaja: combina costo recorrido y heurística, por lo que encuentra soluciones óptimas cuando la heurística es admisible; desventaja: requiere más control y puede consumir más memoria que una búsqueda más simple.